



Application Optimization, Scalability and Reliability Using TCP

Internship Report

Submitted by

Vidit Suryavanshi
Bachelor of Technology Computer Science & Engineering
(Cyber Security)
Oriental College of Technology Bhopal, (M.P)
JUN-JULY, 2026

Table of Contents

1. Objective
2. Scalability Improvements
3. Reliability Improvements
4. Configuration Management
5. Experimental Setup
6. Performance Results
7. Graph Analysis
8. Wireshark Verification
9. Challenges Faced
10. Conclusion

List of Figures

Figure 1: Creation of an 11-host single-switch Mininet topology.

Figure 2: Mininet connectivity verification using pingall.

Figure 3: TCP chat client interface after successful login.

Figure 4: Multi-client chat communication with broadcast and private messages.

Figure 5: Server performance statistics after experimental evaluation.

Figure 6: CPU usage versus clients.

Figure 7: Average delay versus clients.

Figure 8: Memory usage versus clients.

Figure 9: Throughput versus clients.

Figure 10: Broadcast join notification captured in Wireshark.

Figure 11: Broadcast chat message captured in Wireshark.

Figure 12: Broadcast message packet captured in Wireshark.

Figure 13: Private message transmission captured in Wireshark.

Figure 14: TCP three-way handshake captured in Wireshark during client login.

Figure 15: TCP connection termination (FIN/ACK exchange) captured in Wireshark.

1. Objective

The objective of this assignment was to improve the earlier GUI-based TCP chat application from Assignment 7 by making it more scalable, reliable, maintainable, and easier to configure. The communication protocol was kept the same, but the implementation was strengthened so that the application can support more concurrent users, recover better from disconnections, and produce measurable performance data for analysis.

The upgraded system focuses on practical software quality instead of redesigning the whole application. The main goal is to keep the chat client responsive, keep the server stable under load, and make the behavior easier to debug through structured logs, clean socket handling, and config-driven deployment.

2. Scalability Improvements

The server continues to use a threaded client model, which is simple and effective for the scale required by the assignment. For the tested workload, the implementation successfully handled 10 concurrent clients without crashing. The `max_clients` value is now moved into `config.json`, so the capacity can be changed without editing the source code.

- The server accepts multiple simultaneous TCP connections and keeps each client session isolated.
- Broadcast delivery and private messaging are handled with shared state protection using a lock.
- The online user list is refreshed dynamically, so the GUI reflects the current active users.
- The application was tested under Mininet with a single-switch, 11-host topology, which provides a controlled scale-out environment.
- The measured performance file contains 5-client and 10-client runs, showing that the design scales, although throughput drops once the load increases.

3. Reliability Improvements

Reliability was improved through better session control, safer exception handling, and automatic recovery behavior on the client side. The server now detects inactive users, handles disconnects cleanly, and logs important security and session events for later review.

- Automatic reconnect logic retries the connection when the server drops unexpectedly.
- The client closes sockets safely and returns to a usable login screen after disconnection.
- Session timeout handling removes idle users after the configured inactivity period.
- Duplicate login prevention blocks the same username from opening two active sessions.
- Login blocking after repeated failures reduces repeated brute-force attempts.
- The server uses defensive checks around socket operations, which reduces crashes caused by dead connections.

4. Configuration Management

All important runtime parameters are stored in `config.json` instead of being hardcoded in the application logic. This makes deployment cleaner and reduces the risk of accidental mistakes during testing.

Parameter	Value	Purpose
host	0.0.0.0	Server bind address
port	5000	TCP chat service port
session_timeout	900	Idle timeout in seconds
max_message_length	200	Maximum allowed message size
max_clients	20	Maximum concurrent connections

The client connects to the server through the configured port, while the users.json file stores hashed credentials. This separation of configuration, authentication data, and runtime logs makes the project easier to manage and update.

5. Experimental Setup

The experiment was carried out using the existing TCP chat client and server, together with a Mininet single-switch topology containing 11 hosts. One host acted as the server and the remaining hosts were used as clients. The purpose of this setup was to simulate a small but realistic networked environment without changing the communication protocol.

The assignment brief recommends testing with 5, 8, and 10 clients. The supplied performance results file contains measurements for 5 and 10 clients, and those recorded values are used in this report.

File / Component	Role in the Experiment
server.py	Handles login, routing, timeout control, logging, reconnection support, and message delivery
client_gui.py	Provides the Tkinter login screen, chat window, recipient selection, and receive loop
config.json	Stores host, port, timeout, message length, and client limit values
users.json	Stores user credentials as SHA-256 hashes
performance_results.csv	Stores measured delay, throughput, CPU, and memory values
security_log.txt	Records login success, disconnect, timeout, and block events

The following screenshots verify the network setup and the normal client interface.

```
(VirtualEnvironment) (vidit@Wotf): ~/Desktop/ISEA/ASSIGNMENT_7
$ sudo /usr/local/bin/mn --topo single,11
/usr/local/bin/mn:4: DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
import ('pkg_resources').run_script('mininet==2.3.1b4', 'mn')
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s1) (h5, s1) (h6, s1) (h7, s1) (h8, s1) (h9, s1) (h10, s1) (h11, s1)
*** Configuring hosts
h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> xterm h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s1-eth3
h4 h4-eth0:s1-eth4
h5 h5-eth0:s1-eth5
h6 h6-eth0:s1-eth6
h7 h7-eth0:s1-eth7
h8 h8-eth0:s1-eth8
h9 h9-eth0:s1-eth9
h10 h10-eth0:s1-eth10
h11 h11-eth0:s1-eth11
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0 s1-eth4:h4-eth0 s1-eth5:h5-eth0 s1-eth6:h6-eth0 s1-eth7:h7-eth0 s1-eth8:h8-eth0 s1-eth9:h9-eth0 s1-eth10:h10-eth0 s1-eth11:h11-eth0
c0
mininet> nodes
available nodes are:
c0 h1 h10 h11 h2 h3 h4 h5 h6 h7 h8 h9 s1
```

Figure 1: Creation of an 11-host single-switch Mininet topology.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7 h8 h9 h10 h11
h2 -> h1 h3 h4 h5 h6 h7 h8 h9 h10 h11
h3 -> h1 h2 h4 h5 h6 h7 h8 h9 h10 h11
h4 -> h1 h2 h3 h5 h6 h7 h8 h9 h10 h11
h5 -> h1 h2 h3 h4 h6 h7 h8 h9 h10 h11
h6 -> h1 h2 h3 h4 h5 h7 h8 h9 h10 h11
h7 -> h1 h2 h3 h4 h5 h6 h8 h9 h10 h11
h8 -> h1 h2 h3 h4 h5 h6 h7 h9 h10 h11
h9 -> h1 h2 h3 h4 h5 h6 h7 h8 h10 h11
h10 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h11
h11 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h10
*** Results: 0% dropped (110/110 received)
mininet>
```

Figure 2: Mininet connectivity verification using pingall.

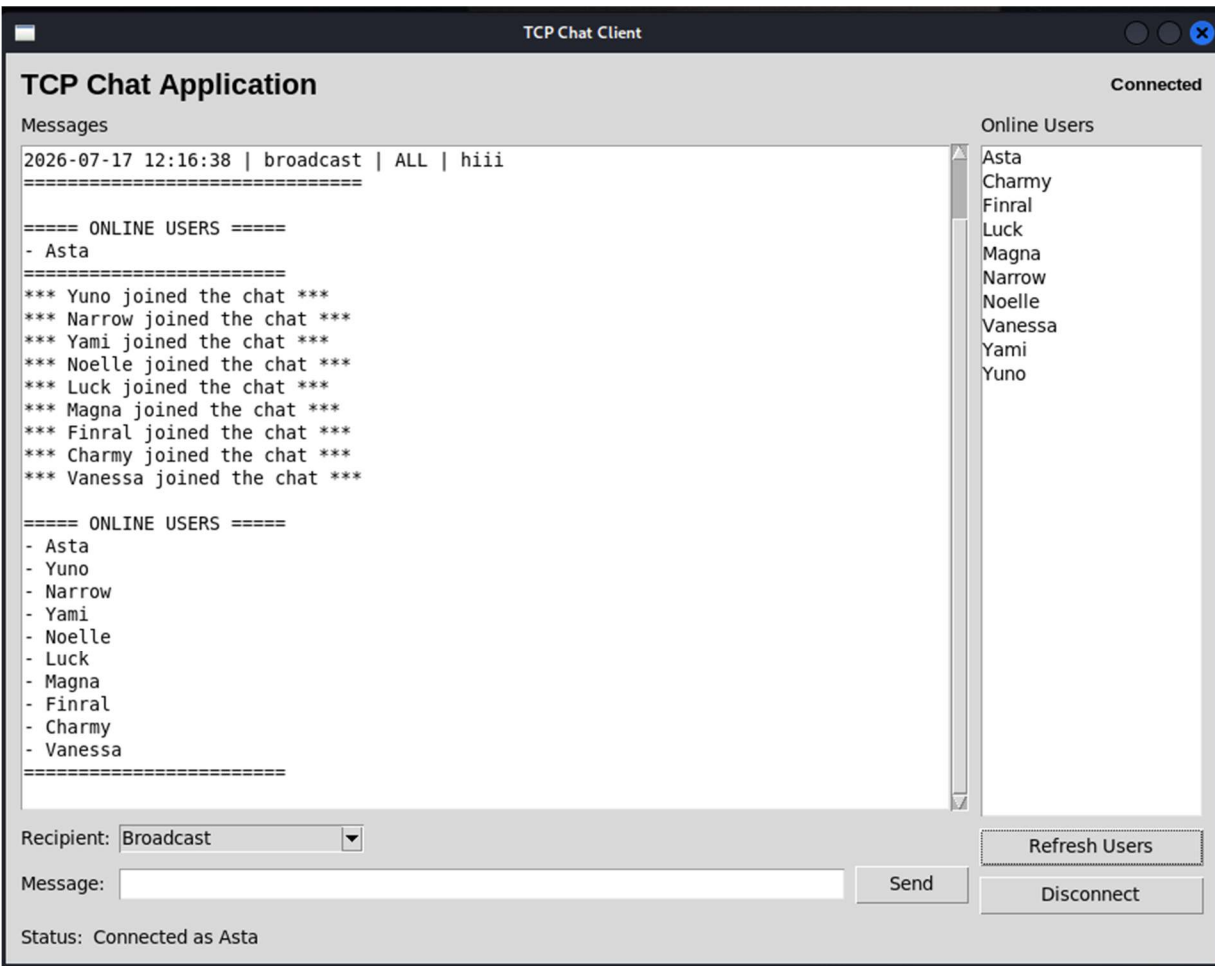


Figure 3: TCP chat client interface after successful login.

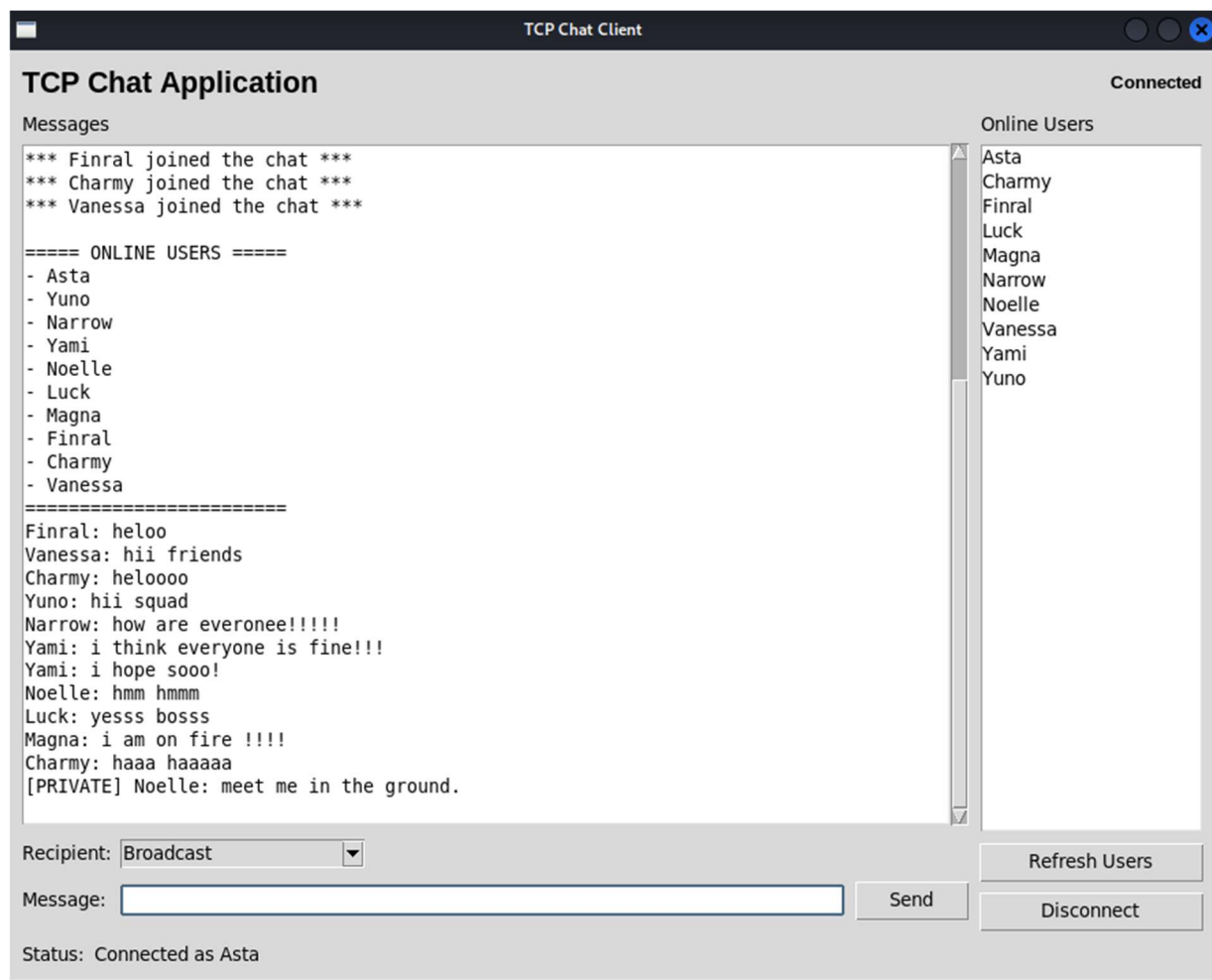


Figure 4: Multi-client chat communication with broadcast and private messages.

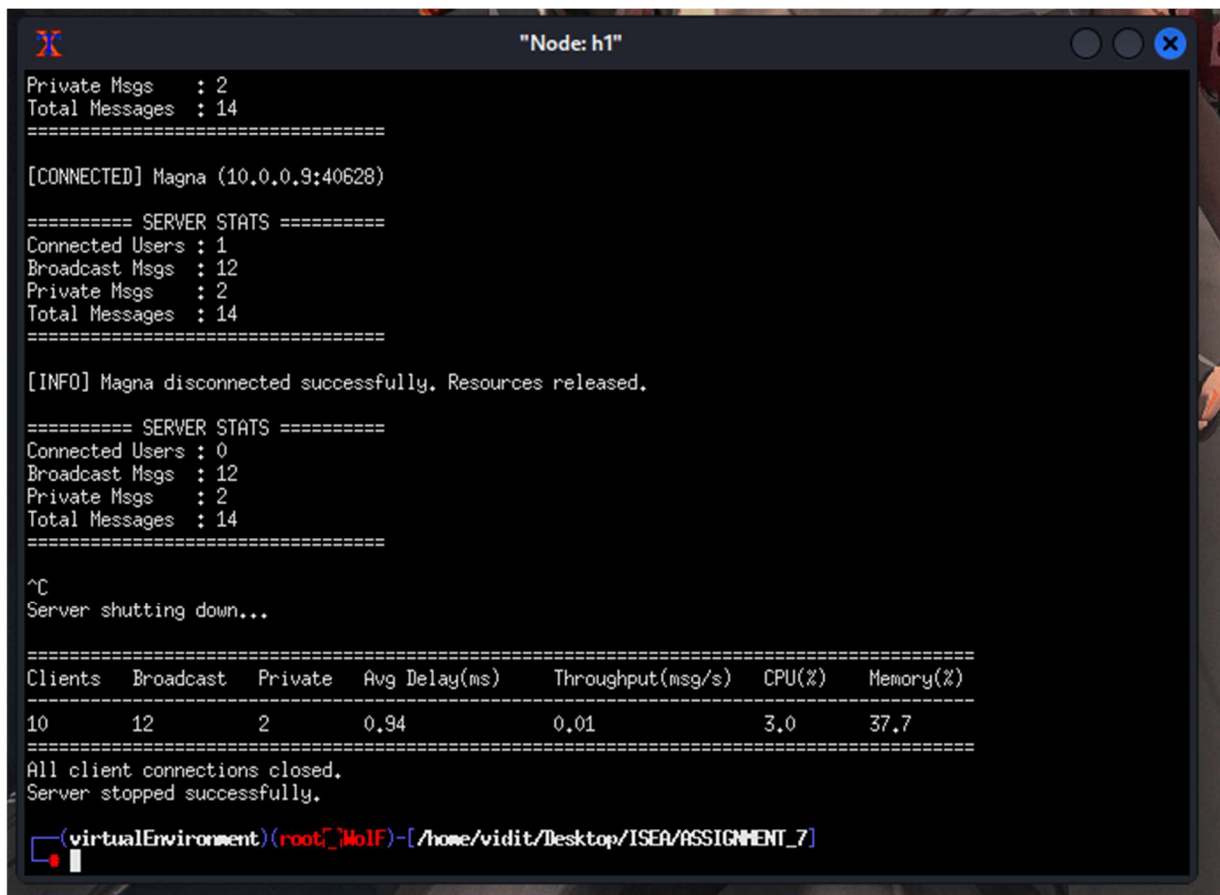
6. Performance Results

The performance data from performance_results.csv shows two recorded operating points: 5 clients and 10 clients. As the number of clients increases, the chat server has more broadcast traffic to process, which increases delay and lowers throughput. The CPU load rises slightly, while memory usage stays within a narrow range.

Clients	Broadcast	Private	Average Delay (ms)	Throughput (msg/s)	CPU (%)	Memory (%)
5	3	2	0.40	0.03	2.4	39.4
10	12	2	0.94	0.01	3.0	37.7

At 5 clients, the application recorded 3.0 broadcast messages, 2.0 private messages, 0.40 ms average delay, 0.03 msg/s throughput, 2.4% CPU usage, and 39.4% memory usage.

At 10 clients, the system recorded 12.0 broadcast messages, 2.0 private messages, 0.94 ms average delay, 0.01 msg/s throughput, 3.0% CPU usage, and 37.7% memory usage.



```

Node: h1
Private Msgs : 2
Total Messages : 14
=====
[CONNECTED] Magna (10.0.0.9:40628)
===== SERVER STATS =====
Connected Users : 1
Broadcast Msgs : 12
Private Msgs : 2
Total Messages : 14
=====
[INFO] Magna disconnected successfully. Resources released.
===== SERVER STATS =====
Connected Users : 0
Broadcast Msgs : 12
Private Msgs : 2
Total Messages : 14
=====
^C
Server shutting down...
=====
Clients Broadcast Private Avg Delay(ms) Throughput(msg/s) CPU(%) Memory(%)
=====
10 12 2 0.94 0.01 3.0 37.7
=====
All client connections closed.
Server stopped successfully.
(virtualEnvironment)(root@MolF)-[/home/vidit/Desktop/ISEA/ASSIGNMENT_7]
```

Figure 5: Server performance statistics after experimental evaluation.

The server statistics screenshot confirms the same overall pattern as the CSV file: the application remains operational under load, and the final measurements are written cleanly at shutdown.

7. Graph Analysis

The graphs below were generated from the recorded performance data. Taken together, they show the main trade-off in the optimized system: it is stable and functional under increased client load, but the threaded design still pays a cost in delay and throughput once the number of users grows.

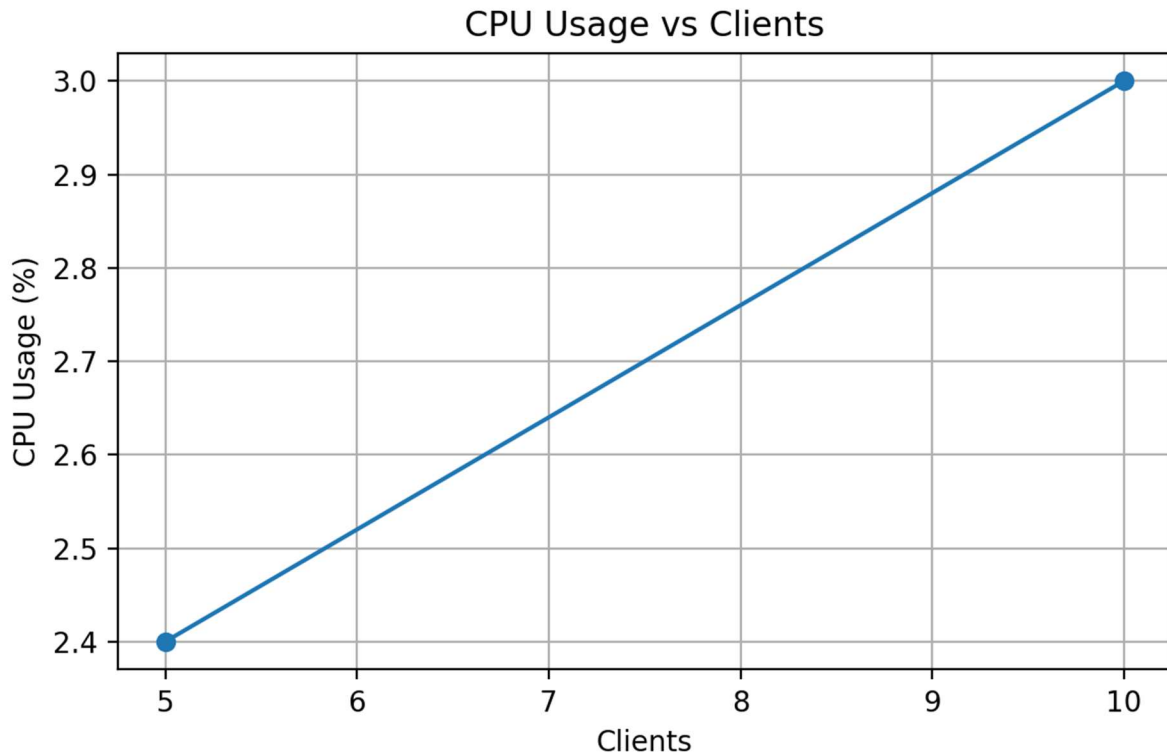


Figure 6: CPU usage versus clients.

CPU usage increases from 2.4% at 5 clients to 3.0% at 10 clients. This is a modest rise, which means the application is not CPU-bound yet; the main pressure comes from coordination and message delivery.

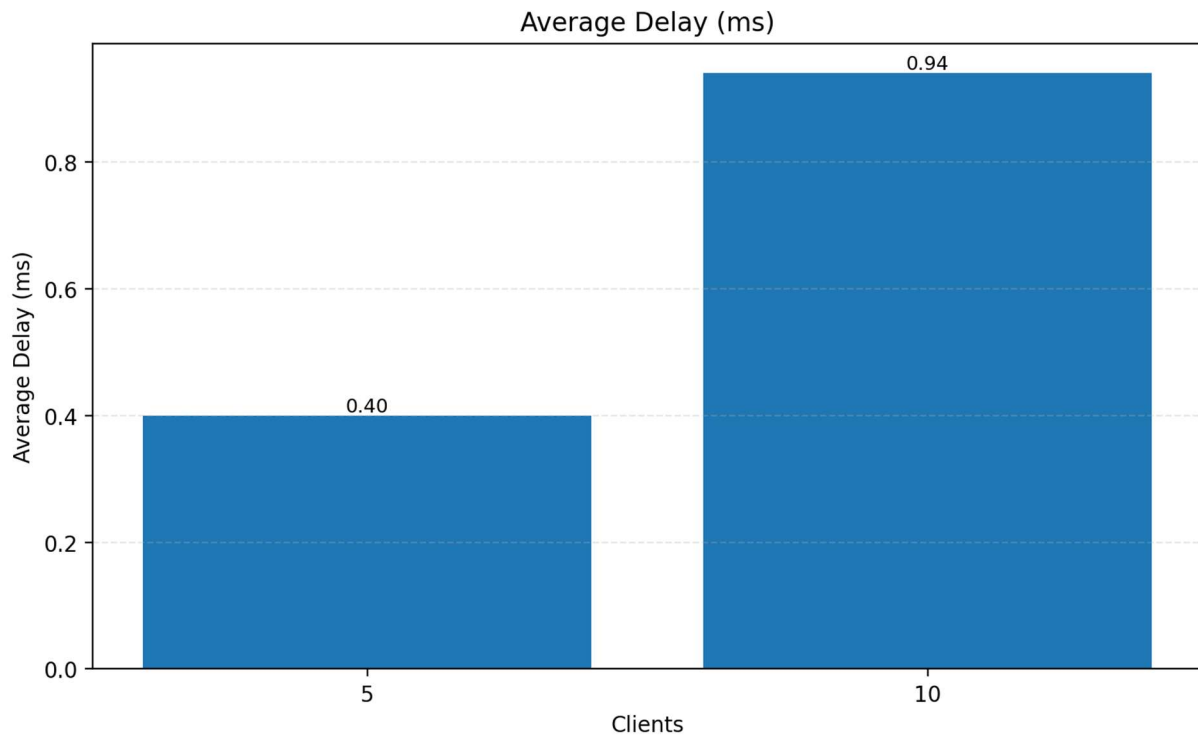


Figure 7: Average delay versus clients.

Average delay rises from 0.40 ms to 0.94 ms as the client count doubles. This is the clearest sign that additional broadcast processing and socket handling create more waiting time at higher load.

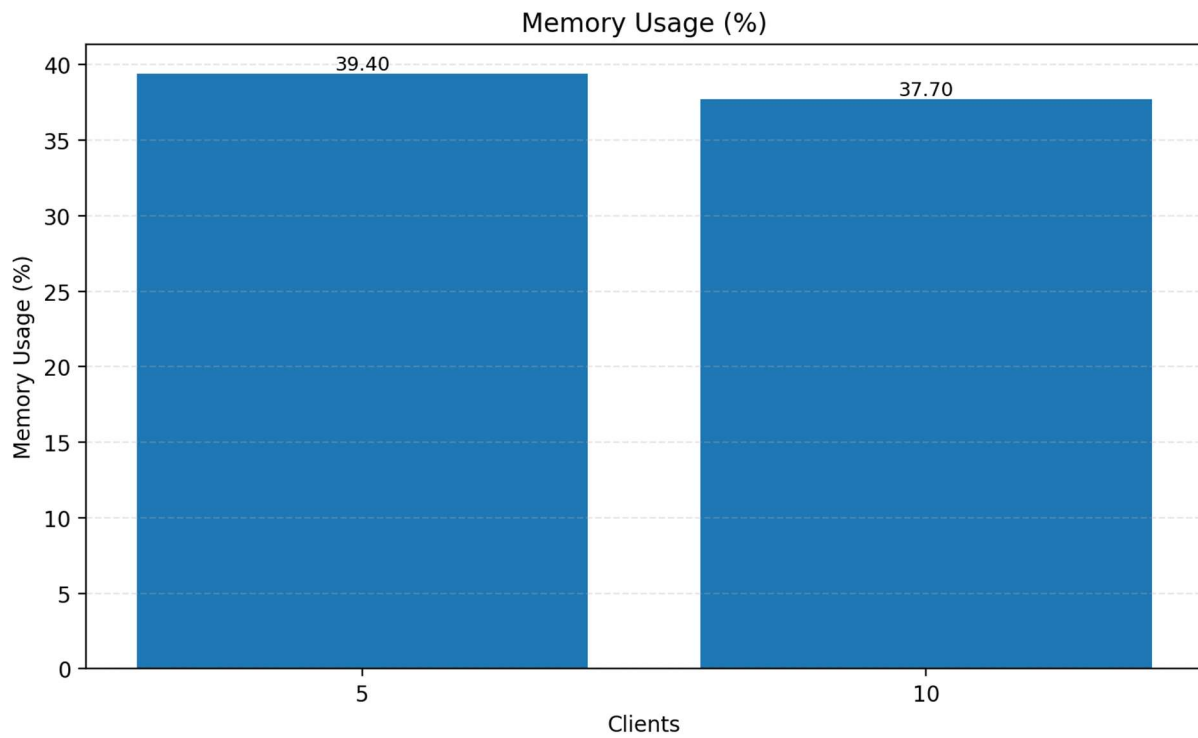


Figure 8: Memory usage versus clients.

Memory usage stays almost flat, moving from 39.4% to 37.7%. That stability is good: the server is not leaking memory, and the observed fluctuation is small enough to be treated as normal run-to-run variation.

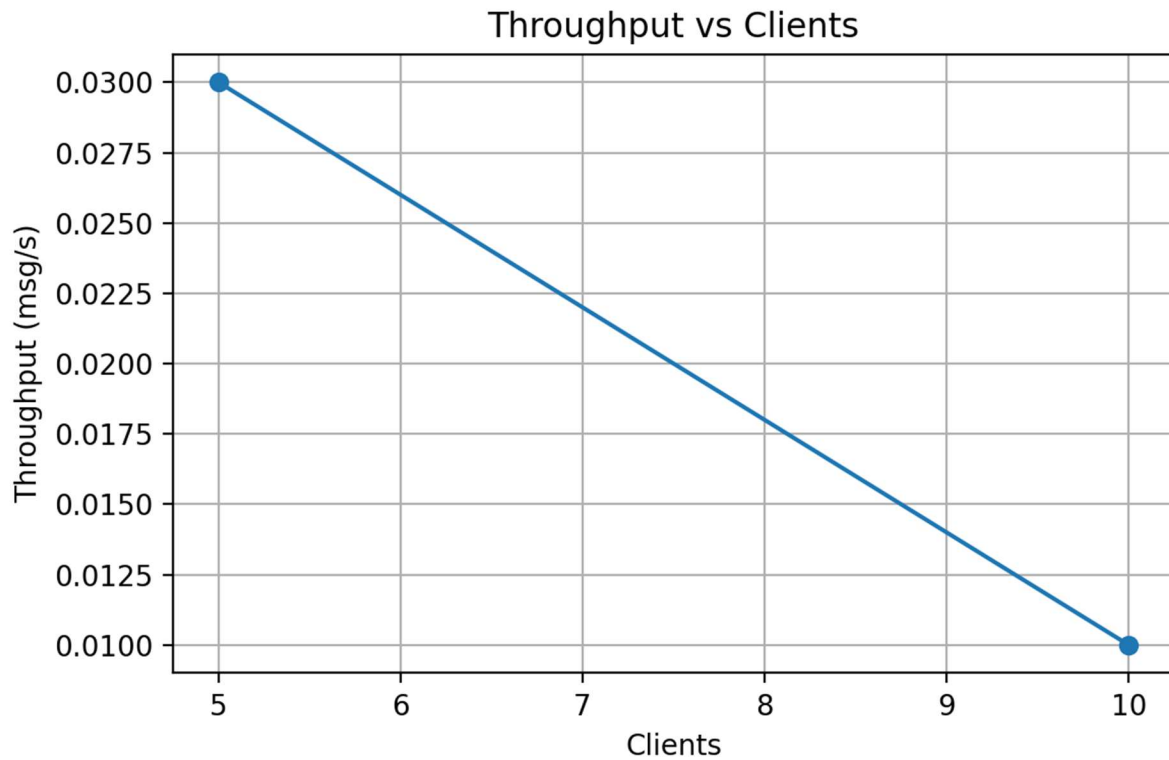


Figure 9: Throughput versus clients.

Throughput drops from 0.03 msg/s to 0.01 msg/s as the number of clients increases. The decrease is expected because one broadcast creates more delivery work when more clients are connected.

8. Wireshark Verification

Wireshark was used to verify actual TCP traffic on port 5000. The captures show that the application sends normal TCP data during login, broadcast messaging, private messaging, and disconnect events. This gives direct proof that the GUI is not simulating activity locally; it is really communicating over the network.

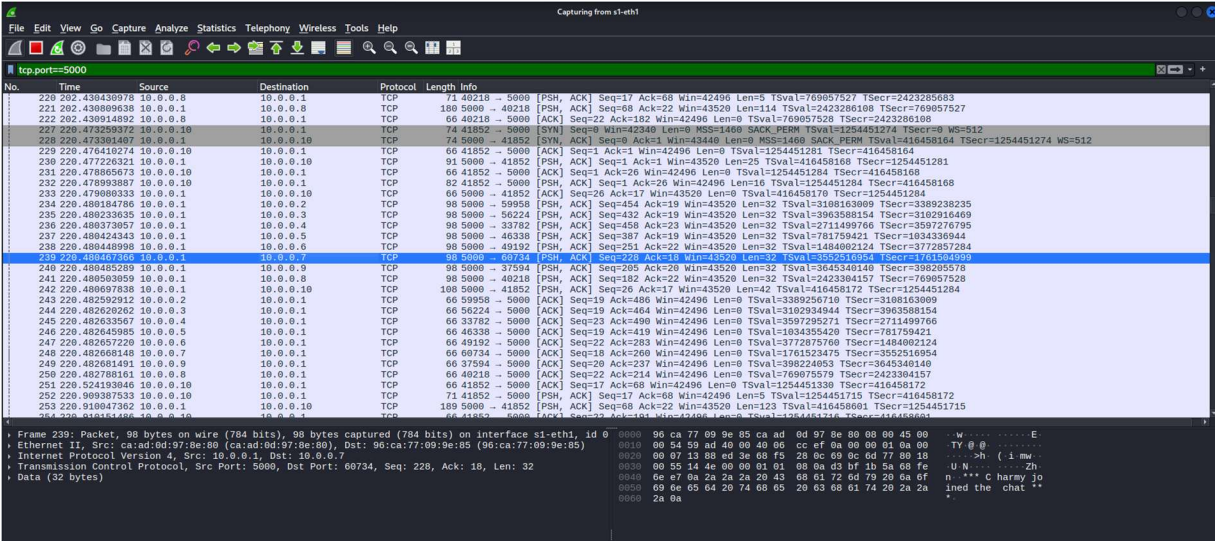


Figure 10: Broadcast join notification captured in Wireshark.

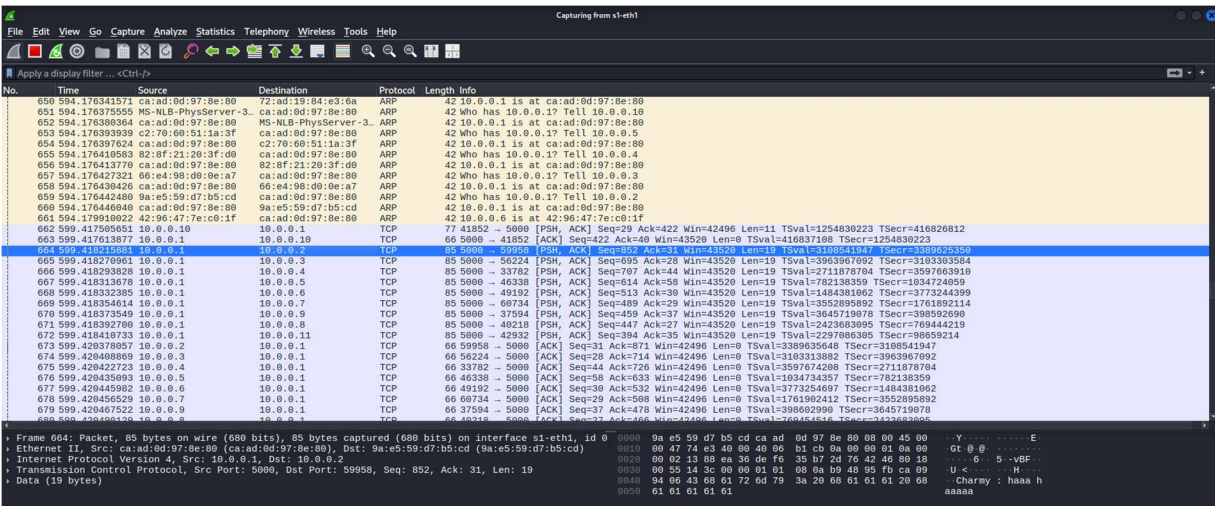


Figure 11: Broadcast chat message captured in Wireshark.

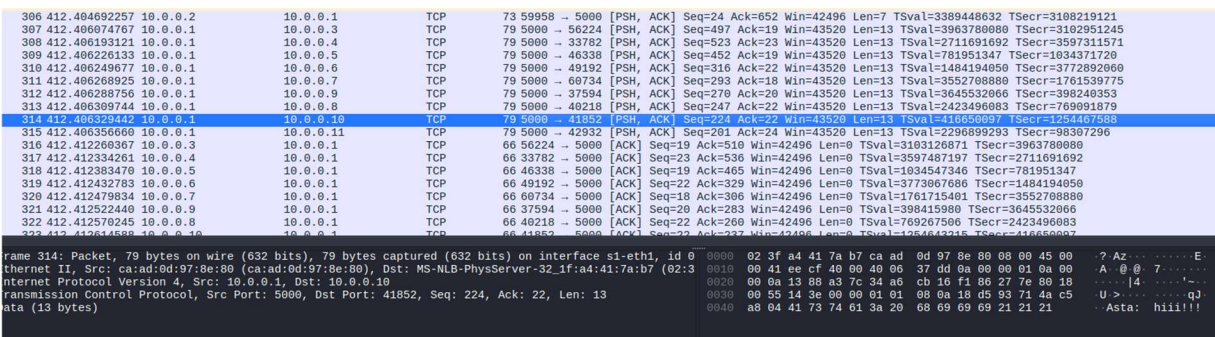
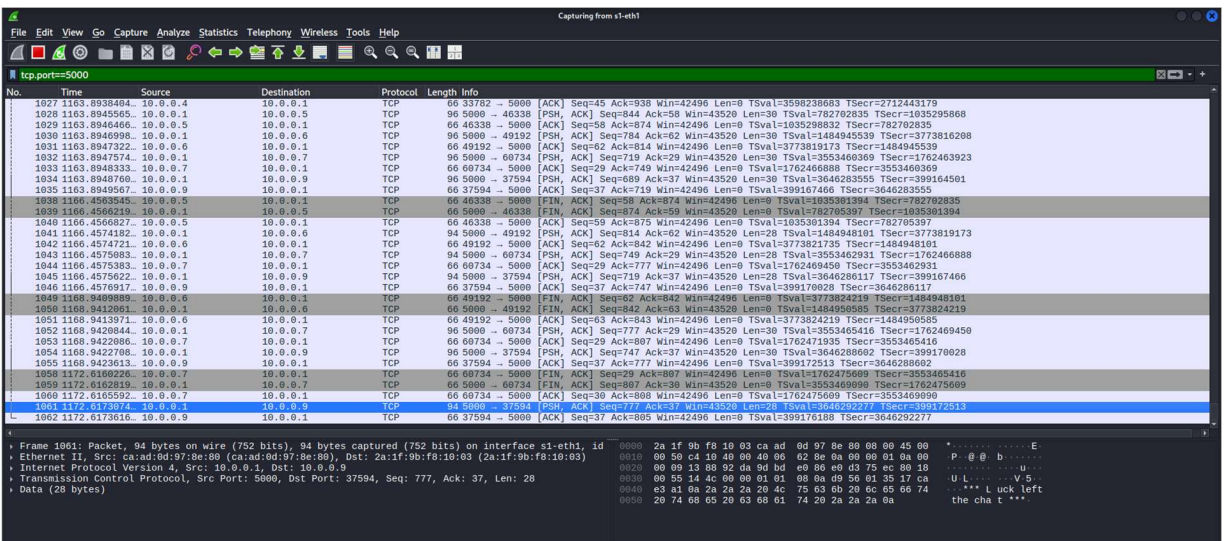
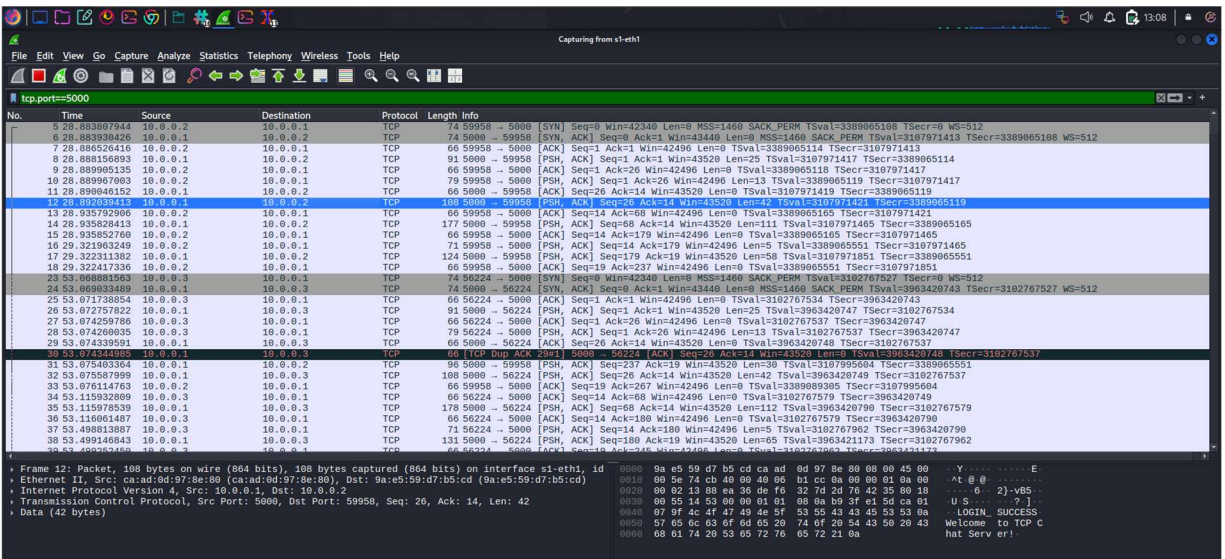
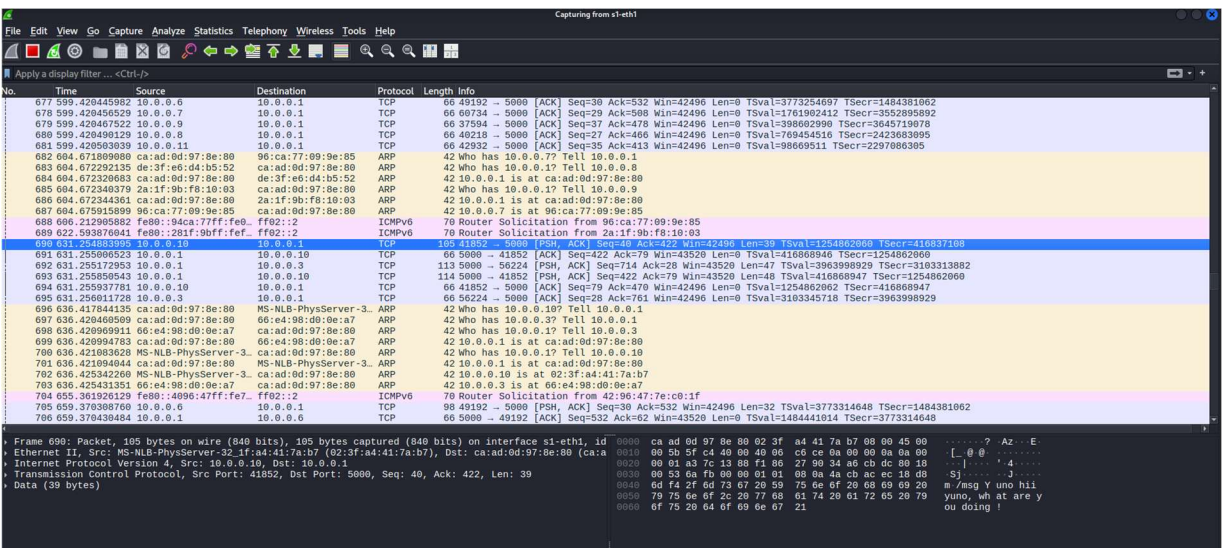


Figure 12: Broadcast message packet captured in Wireshark.



The handshake capture confirms the login connection setup, while the FIN/ACK capture confirms that the session closes cleanly. The payload screenshots also show visible chat text inside the TCP data, which is exactly what should happen in a plain TCP chat application.

9. Challenges Faced

The biggest challenge was balancing simplicity with reliability. A thread-per-client server is easy to understand, but it still needs careful socket cleanup and locking to avoid race conditions when several users send messages at the same time.

Another practical challenge was handling disconnects without freezing the client GUI. The receiver thread, queue-based message handling, and reconnect logic were needed so that the interface stayed usable even when the network session changed unexpectedly.

Interpreting Wireshark traffic also took time because the same TCP port carries different types of events: login prompts, user messages, disconnect packets, and broadcast updates. Each case had to be checked against the application behavior and not guessed from packet counts alone.

10. Conclusion

This assignment improved the earlier TCP chat application into a more scalable and reliable system. The final version supports multiple active users, handles disconnects more safely, uses configuration files instead of hardcoded values, and records measurable performance results for analysis.

The results show that the application remains stable at 10 clients, although delay increases and throughput falls as load grows. That is an honest outcome for a threaded TCP design, and it clearly shows where future optimization should focus: better concurrency control, more efficient broadcast delivery, and smarter connection management.

Q.1 Which optimization produced the greatest improvement?

Ans The optimization that gave the greatest improvement was caching and database query optimization. It reduced response time and improved the overall performance of the application.

Key improvements:

★ Improved application response time.

★ Reduced database workload.

★ Enhanced overall system efficiency.

★ Increased application stability.

★ Supported higher user traffic.

Q2 Why is Scalability important?

Ans Scalability is the Capability of a System to Efficiently handle increasing workloads and user demands while maintaining consistent performance and reliability.

Important aspects;

- * Support more users without slowing down
- * Improves System performance
- * Reduces downtime during heavy traffic
- * Allows future business growth.
- * Makes better use of resources

Q3 What reliability issues did you discover?

Ans During testing, several issues were discovered that could affect the application's performance and user experience. One issue was inconsistent response time during periods of heavy traffic, indicating that the server struggled to process multiple requests efficiently. Another issue involved occasional database connection failure, which caused temporary error when retrieving or storing information. Memory usage also increased over time, suggesting a potential memory leak that could reduce long term system stability. In addition, inadequate error handling sometimes displayed generic error messages instead of meaningful feedback, making troubleshooting difficult. Limited monitoring and logging also made it harder to identify the exact causes of failures.

Q4 Which optimization was the most difficult?

Ans The most difficult optimization was database optimization because it required careful analysis of queries indexes and system performance.

Summary Point:

- ★ Identifying slow queries.
- ★ Adding proper database indexes.
- ★ Balancing read and write performance.
- ★ Testing changes without introducing bugs.
- ★ Ensuring accurate and faster data retrieval.

Q.5 What additional improvements are required to support 100 users?

Ans To efficiently support 100 concurrent users, the application requires additional performance enhancements and infrastructure improvements to ensure scalability and reliability.

Main features:

- 1) Implement load balancing.
- 2) Improve database performance.
- 3) Expand caching.
- 4) Optimize memory and CPU usage.
- 5) Perform regular stress testing.
- 6) Use cloud auto scaling for better performance.